

LBIBCell: A Cell-Based Simulation Environment for Morphogenetic Problems

Simon Tanaka^{1,2*}, David Sichau¹, and Dagmar Iber^{1,2†}

March 4, 2022

Abstract

1 Motivation:

The simulation of morphogenetic problems requires the simultaneous and coupled simulation of signalling and tissue dynamics. A cellular resolution of the tissue domain is important to adequately describe the impact of cell-based events, such as cell division, cell-cell interactions, and spatially restricted signalling events. A tightly coupled cell-based mechano-regulatory simulation tool is therefore required.

2 Results:

We developed an open-source software framework for morphogenetic problems. The environment offers core functionalities for the tissue and signalling models. In addition, the software offers great flexibility to add custom extensions and biologically motivated processes. Cells are represented as highly resolved, massless elastic polygons; the viscous properties of the tissue are modelled by a Newtonian fluid. The Immersed Boundary method is used to model the interaction between the viscous and elastic properties of the cells, thus extending on the IBCell model. The fluid and signalling processes are solved using the Lattice Boltzmann method. As application examples we simulate signalling-dependent tissue dynamics.

*simon.tanaka@bsse.ethz.ch

†dagmar.iber@bsse.ethz.ch

3 Availability:

The documentation and source code are available on <http://tanakas.bitbucket.org/lbibcell/index.html>

4 Contact:

dagmar.iber@bsse.ethz.ch

5 Introduction

During morphogenesis, tissue grows and self-organises into complex functional units such as organs. The process is tightly controlled, both by signalling and by mechanical interactions. Long-range signalling interactions in the tissues can be mediated by diffusible substances, called morphogens, and by long-range cell processes ([35]). The dynamics of the diffusible factors can typically be well described by systems of continuous reaction-advection-diffusion partial differential equations (PDE). The appropriate tissue representation depends on the relevant time scale. For a homogeneous isotropic embryonic tissue, experiments show that the tissue is well approximated by a viscous fluid on long time scales (equilibration after 30 minutes to several hours) and by an elastic material on short time scales (seconds to minutes) ([10]). However, biological control typically happens on a shorter time scale, and many cellular processes such as cell migration and adhesion, cell polarity, directed division, monolayer structures and

differentiation cannot be cast into a continuous formulation in a straight-forward way. A number of cell based simulation techniques at different scales and different level of detail have been developed to study these processes; here, we discuss main representatives for each category.

The *Cellular Potts model*, introduced by [12], is solved on a lattice, with each lattice point holding a generalized spin value denoting cell identity. Similar to the Ising model, Hamiltonian energy functions are formulated and minimized using a Metropolis algorithm. It has been applied to a multitude of problems, and is implemented in the software *CompuCell3D* ([39]). However, the correspondence between the biological problem and the Hamiltonian, the temperature and the time step is not always straightforward.

The *subcellular element model* divides cells into subcellular elements, which are represented by computational particles. The elements interact via interacting potentials which are subject to modelling. The motion of the elements is governed by overdamped Langevin dynamics, such that the method is mesh-free. The framework was first introduced by [25] and later applied by [36, 37]. This approach allows for detailed biophysical modelling, both in 2D and 3D.

The *spheroid model* developed by [7] assumes that cells in unstructured cell populations are similar to colloidal particles. The cells are modelled as point particles, hosting interaction potentials. Their motion consist of a random and a directed movement. Neighboring cells form adhesive bonds, which are represented using models borrowed from contact mechanics, such as e.g. the Johnson-Kendall-Roberts model ([6]). Many cellular processes such as cell shape change, division, death, lysis, cell-cell interaction and migration have been successfully translated into the spheroid model ([7]). Intra- and extracellular diffusion has not yet been introduced and implemented. The spheroid model extends efficiently to 3D, and it has been implemented in the open-source framework *CellSys* ([15]).

The *vertex model* uses polygons (or polyhedra in 3D) to represent cells in densely packed tissues, e.g. in *Drosophila* wing disc epithelia ([8]). For each vertex, forces are computed - either via a potential or

directly. The vertices are moved subsequently according to overdamped equations of motion, or via a Monte Carlo algorithm. The model is implemented in the open-source software *Chaste* ([31]).

The *viscoelastic cell model* (also called IBCell models) presented in [34] and [33] uses the immersed boundary method ([29]) to represent individually deformable cells as immersed elastic bodies. The cytoplasm and the extracellular matrix and fluid are represented by a viscous incompressible fluid. In this framework, a vast amount of biological processes such as cell growth, cell division, apoptosis and polarization has been realized. The model was applied to study tumor and epithelial dynamics. Due to the very high level of detail, the viscoelastic cell model is computationally expensive and has not yet been implemented in 3D.

The software framework *VirtualLeaf* with explicit cell resolution, available in 2D, has been introduced in [22]. Although the cell representation is similar to vertex cell models, the dynamics is realized by minimizing an Hamiltonian using a Monte Carlo algorithm. The model assumes rigid cell walls, which is appropriate for plant morphogenesis.

For many morphogenetic phenomena, which arise from a tight interaction between the biomolecular signalling and the tissue physics, an explicit computational representation of the cell shapes is required. Here, we present a flexible software framework based on the IBCell model, which, as a novelty, permits to tightly couple biomolecular signalling models to a cell-resolved, physical tissue model. The core components and the general approach of the model are described in the second section. In the third section, the software and the main functionalities are described in detail. Application examples are given in the fourth chapter to demonstrate the framework's capabilities.

6 Approach

Our approach permits the coupled simulation of tissue and signalling dynamics. To describe the tissue dynamics, the visco-elastic cell model needs to represent both the cellular structures and their elastic

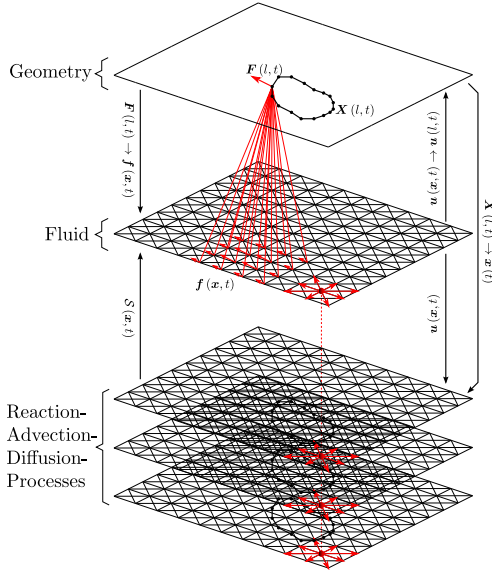


Figure 1: **Algorithm Overview.** The algorithm consists of three coupled layers. The geometry $\mathbf{X}(l, t)$ (top part, discussed in more detail in Figure 2) is used to compute the forces $\mathbf{F}(l, t)$ acting on each of the geometry nodes. These forces, which do not necessarily coincide with a lattice point, are scattered to the fluid lattice (middle part) using the immersed boundary method kernel function, $\mathbf{F}(l, t) \rightarrow \mathbf{f}(t)$. After advancing the fluid solver by one time step, the velocity is interpolated to the geometry node position using the same kernel function, $\mathbf{u}(\mathbf{x}, t) \rightarrow \mathbf{U}(l, t)$. The geometry nodes are moved according to their velocity $\mathbf{U}(l, t)$ and the iteration is restarted. The velocity $\mathbf{u}(\mathbf{x}, t)$ of the fluid lattice is also copied to the reaction-advection-diffusion solvers (PDE), together with the position $\mathbf{X}(l, t) \rightarrow \mathbf{x}(t)$ of the geometry. The state of the reaction-advection-diffusion solvers, which are used to model signalling, may be used to compute mass sources $\mathcal{S}(\mathbf{x}, t)$ for the fluid solver.

properties, as well as the viscous behaviour of the cytoplasm and of the extracellular space surrounding the cells. The model therefore rests on three core parts: the representation of cells, the representation of the fluid and the fluid-structure interaction, and the coupling of the tissue part to the signalling model.

To describe the interaction between the viscous fluid and the elastic structures, which are immersed in the fluid, we use the immersed boundary (IB) method ([29]) as previously implemented in the visco-elastic cell model, also called IBcell model ([34, 33]). To solve the viscous fluid behaviour, we use the Lattice Boltzmann method, which is an efficient mesoscopic numerical scheme, originally developed to solve fluid dynamics problems ([5]). The method has previously been successfully applied to reaction-diffusion equations such as Turing systems ([32]), as well as to coupled scalar fields such as temperature ([13]). The method was for the first time combined with the immersed boundary method ([29]) by [9], and has later been used to study red blood cells in flow by [42]. In the following, we provide an overview of the implemented methods; the implementation details are given in section 7.

6.1 Cell Representation

Cells are represented as massless, purely elastic structures, which are described by sets of geometry points forming polygons. The geometry points are connected via forces. In a first approximation, the elastic structures can be identified to represent the elastic cell membranes. However, more elastic structures can be added to the intra-/extracellular volume to mimic the visco-elastic properties of the cytoskeleton or the extracellular matrix. The user can implement biological mechanisms which operate on the cell representations. For example, a new junction to a neighboring cell might be created when the distance between two neighboring cell boundaries falls below a threshold distance. Similarly, a junction might be removed when overly stretched.

6.2 Fluid and Fluid-Structure Interaction

The visco-elastic cell model represents the content of cells (the cytoplasm) as well as the extracellular space (the interstitial fluid and the extracellular matrix) as a viscous, Newtonian fluid. The intra- and extracellular fluids interact with the elastic membrane, i.e. the fluids exert force on the membrane, and the

membrane exerts force on the fluids. Furthermore, the velocity field of the fluid, which is induced by the forces, moves and deforms the elastic structures. This interaction, well-known as fluid-structure interaction, lies at the heart of the tissue model. Forces (e.g. membrane tension or cell-cell forces) acting on these points are exerted on the fluid by distributing the force to the surrounding fluid. Due to the local forcing, the fluid moves. At this step, the membrane point are advected passively by the fluid. As a result the forces need to be re-evaluated on the points. By repeating the forcing-advective steps, the interaction is realized iteratively.

As a result of this iterative process, the (elastic) structures are coupled to the (viscous) fluid. Depending on the parametrisation, this model allows to describe both elastic, or viscous, or visco-elastic material behaviour. The upper part of Figure 1 illustrates the immersed boundary interaction. The implemented IB kernel function has bounded support, i.e. each geometry point influences and is influenced only its immediate neighborhood. Here, the dimension of the kernel function is four by four (cf. Figure 1). The fluid equations are solved using the Lattice Boltzmann method ([5]), which is described in detail in the supplementary material (section 10). The Reynolds number is typically $\ll 1$, hence the regime is described by Stokes flow¹.

6.3 Signalling

The signalling network is represented as a system of reaction-advection-diffusion processes. The elastic membranes may act as no-flux boundaries for compounds which only exist in the extra- or intracellular volume, respectively. The reaction-advection-diffusion solvers can be equipped with potentially coupled reaction terms in order to model signalling interactions of diffusing factors. Depending on the model, the signalling may impact the tissue dynamics. This can be done, for instance, by making the

mass source of the fluid dependent on the values of the reaction-advection-diffusion solvers such that the tissue expands locally (cf. Figure 1). Furthermore, the diffusing compounds can be individually configured to diffuse freely across the entire domain, or only inside or outside the cells (e.g. using no-flux boundary conditions for the cell membranes).

7 Software

7.1 Cell Representation

The cell geometries consist of two elements, the **GeometryNodes**, which act as the IB points, and the **Connections**, connecting pairs of **GeometryNodes**. A simplified cell is visualized in Figure 2. The **Connections** are attributed with a **domainID** flag, which is an identifier for the surrounded domain (respecting the counter-clockwise directionality convention). The domain identifier on the other side (on the right hand side) is zero by convention, representing the interstitial space. The **domainID** of the **Connections** are copied to the fluid and reaction-advection-diffusion solvers. Moreover, the **domainID**'s are associated with a cell type flag, **cellType**. By applying custom differentiation rules, the **cellType** of individual cells may be changed according to custom criteria; otherwise the all cells default to **cellType**=1 (with **cellType**=0 being the interstitial space, again). In this way, the reaction terms and the mass sources may be made dependent on specific cells, or specific cell types.

7.2 User-provided Solvers

The user can add the following routines: **MassSolverXX**, **CDESolverXX**, and **BioSolverXX** (XX being a name to be chosen). The **MassSolverXX** - as described above - adds or subtracts mass from/to the fluid solver. The **CDESolverXX** is used to implement the reaction terms of the signalling models. Finally, the **BioSolverXX** can be used to execute biologically motivated operations on the geometry and the forces. Such an operation might be cell division, which is discussed in more

¹The Reynolds number reads $Re = \frac{UL}{\nu}$, with U being a characteristic velocity, L a characteristic length scale, and ν the kinematic viscosity. Assuming $L = 10^{-3} [m]$, $U = 10^{-8} [m/s]$ and $\nu = 10^1 \dots 10^2 [m^2/s]$, then $Re = 10^{-13} \dots 10^{-12}$ can be estimated ([10]).

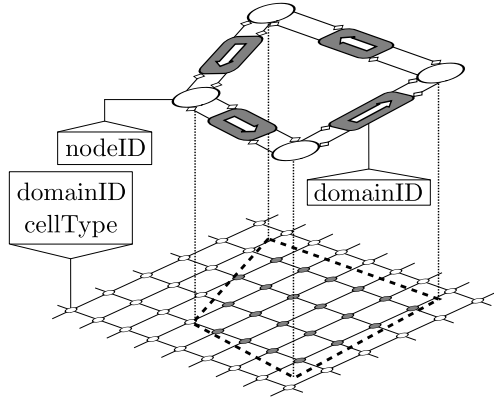


Figure 2: Elements of the Geometry Representation. The cells are closed polygons, consisting of geometry nodes (discs in the top part) and connections (shaded boxes in the top part) between each two geometry nodes. Each connection stores two references to its preceding and successive geometry nodes, and *vice versa* each geometry node stores two references to its preceding and successive connection (visualized by aggregation arrows in the top part). Directionality of the polygon is counter clockwise by convention. Each geometry node has a unique, immutable **nodeID** attribute, which is allocated internally upon creation of a new geometry node. Each connection features a **domainID** attribute, which denotes the domain identifier of the domain on the left hand side. The domain identifier on the right hand side is by definition zero, representing the extracellular space. Using the **domainID** of the connections, the **domainID** of the lattice nodes is automatically set (lower part). Additionally, each **domainID** is associated with a **cellType**. The behaviour of the **MassSolverXX**, **BioSolverXX** and **CDESolverXX** can be made dependent on the **domainID** and/or **cellType** attributes by the user.

detail in Subsection 7.5.4. Figure 3A summarizes the most important classes and their interactions. The classes which are subject to customization are shaded. In order to add a new customized routine (e.g. a mass modifying solver **MassSolverXX**, a reaction-advection-diffusion solver **CDESolverXX**, or

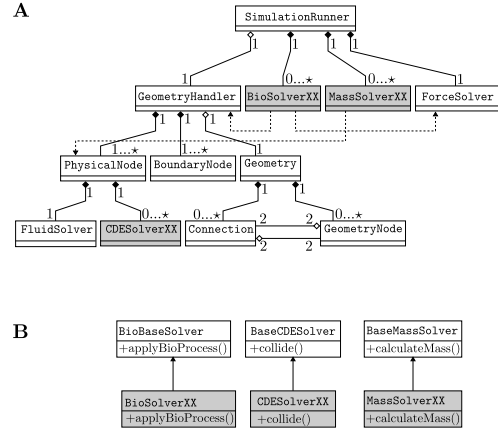


Figure 3: Simplified UML diagram of important Classes. The classes which have to be provided by the user are shaded. **XX** refers to an arbitrary solver name. **A** The **SimulationRunner** controls the execution of the simulation. The **GeometryHandler** has a collection of **PhysicalNodes**, representing the lattice, a collection of **BoundaryNodes** which are woven into the lattice, and a **Geometry** object. The latter contains the cell's geometric information, namely the **GeometryNodes** and the **Connections**. The **GeometryNodes** and the **Connections** each have two references of the preceding and successive elements, as also explained in Figure 2. **BioSolverXX** obtains references from the **GeometryHandler** and the **ForceSolver** to alters states accordingly. Similarly, the **MassSolverXX** obtains a reference to the lattice and adds mass sources to the fluid. **B** To implement new custom routines, the user must inherit from provided base classes (from **BioBaseSolver** for biologically motivated routines, from **BaseCDESolver** for reaction-advection-diffusion processes, and from **BaseMassSolver** for mass modifying routines)

a biologically motivated solver **BioSolverXX**), the user needs to inherit from their respective virtual base classes (cf. Figure 3B). Figure 4 visualizes the routines, which are called iteratively by the **SimulationRunner** (cf. Figure 3A).

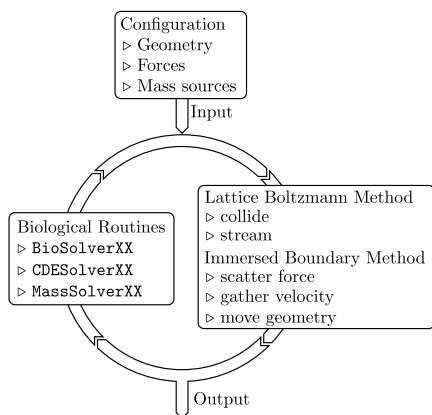


Figure 4: **Iterative Processing in the Solver** At initiation, the library loads the user-provided configuration files (containing global simulation parameters, initial geometry, initial forces). During each iteration, the library’s class `SimulationRunner` (cf. Figure 3A) successively calls the physical routines (the Lattice Boltzmann method to solve the fluid and reaction-advection-diffusion processes, and the immersed boundary method to solve the fluid-structure interaction) and the biological routines (biologically motivated re-arrangement of the geometry, modifications of the forces, etc.). The current configuration and optionally the entire solver states can be saved at a chosen frequency.

7.3 Input and Output

The communication to the user is achieved via the loading and dumping of configuration files. A general configuration file contains the global simulation parameters, such as the simulation time, the domain size, the fluid viscosity, and the diffusion coefficients for the reaction-advection-diffusion solvers. The geometry points and the corresponding geometrical connections are stored in a geometry file. A third file contains the forces, including forces between a pair of geometry points, freely defined forces or spatially anchored points. The fluid and reaction-advection-diffusion solver states may be written either to .txt files or in .vtk format and can be post-processed with third-party software (e.g. Matlab or

ParaView). Optionally, the solver states can be saved in a loadable format to resume the simulation.

7.4 Physical Processes

7.4.1 Viscous and Elastic Behaviour

The viscous behaviour is implemented using a representation of an incompressible fluid (solved using the Lattice Boltzmann method, cf. supplementary material), which converges to the Navier-Stokes equation in the hydrodynamic limit. The fluid is solved on a regular Cartesian and Eulerian grid. The membranes are represented by sets of points, which are connected to form closed polygons. A variety of forces may act on the membrane nodes, such as e.g. membrane tensions (cf. Subsection 7.4.3). The interaction between the fluid and the elastic structures is formulated using the Immersed Boundary method (cf. supplementary material). The membrane points move according to the local fluid velocity field in a Lagrangian manner.

7.4.2 Reaction-Diffusion of Biochemical Compounds

The biochemical signalling can be described by sets of coupled reaction-diffusion partial differential equations. Similar to the fluid equations, these equations are solved on a regular Cartesian and Eulerian grid (solved using the Lattice Boltzmann method, cf. supplementary material). The concentrations of the compounds can be accessed by other solvers, for example to make other processes such as cell division dependent on signalling factors. The cell boundaries can be chosen to be either invisible to the diffusing compounds, or to be no-flux boundaries. To account for advection, the fluid velocity field is directly transferred from the fluid solver since the fluid and the reaction-diffusion lattices coincide spatially. The coupling of the solvers is visualized in Figure 1.

7.4.3 Forces

Forces are an integral part of the simulation environment. A force is always connected to a membrane point. Any type of conservative force (which can be derived from a potential) can easily be implemented.

Currently, the following types of forces are implemented:

- spring force between two geometrical nodes
- spring force between a geometrical node and a spatial anchor point
- free force acting on a geometrical node
- horizontally or vertically sliding force (thus enforcing only the y or x coordinate, respectively)
- constant force between two geometrical nodes

Application examples include constant forces between two geometrical nodes that can be used to model constant membrane tension, which leads to the minimization of a cells perimeter (discussed in Subsection 7.5.2). Moreover, a geometrical point can dynamically explore its local neighbourhood and establish a force to another geometrical point *from another cell*, thus mimicking cell-cell junctions (discussed in Subsection 7.5.3).

7.5 Biological Processes

The biological solvers (BioSolver) accommodate the functionalities that are related to biological processes. These processes may be mostly related to modifications of the forces and the geometry. The BioSolver has full access to the compound concentrations. Furthermore, it is aware of the cells, whose geometries are stored individually. This enables the BioSolver to compute cell areas and averaged or integrated compound concentrations. Since all cells are individually tagged, cell behaviour can be made dependent on cell identity. Additionally to the cell identity, cells also carry a cell type tag, which can be changed depending on run-time conditions. This latter functionality can be used to model cell differentiation.

Consider a cell division event as an example. Here, a division plane has to be chosen. The choice of its position and direction is subject to the user's model: the cell division plane might be set perpendicular to the cell's axis of strongest elongation. Next, the cell has to be divided, which requires the removal of the

corresponding geometrical connections, and the insertion of new geometrical nodes and connections to close the divided cells.

Note that the concentration fields of the compounds, as well as the velocity- and pressure fields of the fluid solver are not directly altered in the biological module.

7.5.1 Control of Cell Area

Depending on the biological model of the user, the cell area has to be controlled. By assuming that a cell might change its spatial extent in the third dimension, the area might shrink or expand as a response to forces exerted by its neighbouring cells, which can effectively be modelled as an 'area elasticity'. In the limiting case, the cell resists external forces, maintains its area and only reacts with changes of the hydrostatic pressure. In general, to control the area of cells, the reference area for each cell needs to be adapted. The reference area acts as a set point for a simple proportional controller, i.e. the local mass source $\mathcal{S}_k$ in the cell k is proportional to the area difference between the current cell area $A_k(t)$, and the set point area A_k^0 :

$$\mathcal{S}_k = \alpha (A_k^0 - A_k(t)) \quad (1)$$

where α is a proportional constant. More advanced control methods, such as e.g. proportional-integral control methods, can be realized easily.

This approach of controlling the cell area can also be used to let cells grow or shrink in a controlled way, i.e. a cell differentiating into an hypertrophic cell type may grow in volume. Implementing this process would be as simple as setting the new target area as set point area. The area controller will bring the cell close to its new area.

7.5.2 Membrane Tension

The definition of forces acting between pairs of membrane points allows for simulating the cell's membrane tension. By default, a constant contracting force $\mathbf{F}_i$ with magnitude φ^{m} is applied to every pair of neighbouring membrane points. Hence the resulting force on membrane point i is composed of a force

pointing to its preceding membrane point $i-1$, and a force pointing to its successive membrane point $i+1$:

$$\mathbf{F}_i^m = \varphi^m \left(\frac{\mathbf{x}_{i-1} - \mathbf{x}_i}{|\mathbf{x}_{i-1} - \mathbf{x}_i|} + \frac{\mathbf{x}_{i+1} - \mathbf{x}_i}{|\mathbf{x}_{i+1} - \mathbf{x}_i|} \right) \quad (2)$$

This approach can be interpreted as an actively remodelled membrane: when stretched, new membrane is synthesized in order to not increase the membrane tension on longer time scales (hours). On the other hand, excessive membrane is degraded to abide the membrane tension. Therefore, the membrane tension minimizes the cell's perimeter. Since the intracellular fluid (and thus the cell area) is conserved in the absence of neighbouring cells and active mechanisms (c.f. Subsection 7.5.1), the cell assumes a circular shape. On short time scales (seconds), the passive (non-remodelled) elastic membranes can be modelled by using Hookean spring potentials. The membrane tension will then be proportional to deviation from the resting membrane perimeter. In both cases, the membrane is flexible (i.e. has no bending stiffness); if bending stiffness should be required by the user, this can be easily realised in a custom `BioSolver`.

The implementation of membrane tension needs to consider the geometry remeshing. Whenever a new membrane point is inserted, it needs to get connected to its neighbours instantly, because the cell will be overly stretched in the absence of membrane tensions. A membrane point's forces need to be removed upon its removal. Algorithmically, this is realized by removing and reconstructing all membrane forces at every time step. At this point, the magnitude of the membrane tension can be made dependent on signalling factors.

`BioSolverMembraneTension` is an example of a class managing the membrane tensions with immediate remodeling, and `BioSolverHookeanMembraneTension` implements simple Hookean springs.

7.5.3 Cell Junctions

A cell can create cell junctions to neighbouring cells. In the simplest case, each membrane point i uses the function `getGeometryNodesWithinRadiusWithAvoidance-`

`Closest` to get the closest membrane point j of another cell, which is within a predefined cut-off radius l^{max} , or zero if there is no such membrane point. Once a candidate membrane point fulfils the criteria, a new Hookean force $\mathbf{F}_i$ with a spring constant k^j and resting length l_0 is created:

$$\mathbf{F}_i^{cj} = \begin{cases} k^j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|} (|\mathbf{x}_j - \mathbf{x}_i| - l_0) & \text{if } |\mathbf{x}_j - \mathbf{x}_i| < l^{max} \\ 0 & \text{else} \end{cases} \quad (3)$$

The cell junction forces are regularly (potentially not at every time step) deleted and renewed, where the frequency of cell junction renewal might reflect the cell junction synthesis rate.

The function `getGeometryNodesWithinRadius-WithAvoidance` returns all membrane points of another cell, which are within a predefined cut-off radius; the returned list might be empty. This opens up the possibility to introduce randomness by choosing the membrane point randomly from the candidate list. The probability to create a junction might depend on the junction length: the shorter, the higher the probability to form a new junction. Also the removal of membrane points might be randomized, and the probability made dependent on the junction length, i.e. overly stretched junctions are removed with higher probability. Even the membrane point whose junctions shall be updated might be chosen randomly. Again, the number of updated membrane nodes per time reflects the cell's limited cell junction synthesis activity.

The membrane points are internally stored in an fast neighbor list data structure, which is well suited for spatial range queries. `BioSolverCellJunction` is an example of a class responsible for cell junctions.

7.5.4 Cell Division

The cell division functionality requires several steps. First, criteria will have to be defined which cells shall be divided. Criteria might be maximal cell area, maximal spatial expansion, or biochemical signals. Once a cell committed for division, the cell division plane will have to be chosen. Again, how to chose the plane is subject to biological modelling. A frequently used rule is to use a plane defined by a random direction

vector and the center of mass of the cell. However, different rules can be readily implemented, such as random directions drawn from non-uniform probability distributions (which, in turn, can be controlled e.g. by signalling factor gradients) or division planes perpendicular to the longest axis ([23]). In a next step, the two membrane segments are determined which intersect with the division plane; this is implemented in `getTwoConnectionsRandomDirection` or `getTwoConnectionsLongestAxis`. These two membrane segments are subsequently removed, and two new membrane segments across the cell are introduced, leading to a cut through the mother cell. Finally, a new domain identity number has to be given to one of the daughter cells; the other daughter cell inherits the domain identity number from the mother cell. The new domain identity number is set to the largest domain identity number plus one, and it is automatically copied to the physical grid. Both daughter cells by default inherit the cell type flag from the mother cell, which is also automatically copied to the physical grid.

The basic cell division functionality is implemented in the class `BioSolverCellDivision`.

7.5.5 Differentiation

Differentiation changes the cell type flag of the cells according to user-defined, biologically motivated rules. These rules might be based on the cell area, or on a signalling factor concentration, possibly integrated over the cell area. Once being committed for differentiation, the cell changes its cell type flag according to the rule. The new cell type flag will be automatically copied to the physical grid. The cell type flag can be used to make signalling dynamics, but also other biologically motivated processes dependent on the cell type.

The association between the domain identifier flags and the cell type flags is stored in the `cellTypeTrackerMap`, which is a member of the `GeometryHandler`. This makes sure that all `BioSolverXX` classes have easy access to this information. A basic implementation of the differentiation control can be found in `BioSolverDifferentiation`.

7.6 Accuracy and Performance

The Lattice Boltzmann schemes are second order accurate, and the explicit immersed boundary method is first order accurate in space and time. The internal data structure uses a fast neighbor list (cell list) implementation to optimize for range queries (e.g. searching for other cells in the local neighborhood), which exhibits a search complexity of $\mathcal{O}(N)$, with N being the number of membrane points to represent the cells. Many iterative computations (LB and IB routines such as particle streaming and collision, gathering of velocity and scattering of force) are parallelized using the shared memory paradigm. However, a few computational steps cannot be parallelized. This is typically the case when write-operations occur on shared data structures, such as the data structures storing the geometry nodes and the force structs (e.g. in `ForceSolver::deleteForceType()` and `GeometryHandler::computeAreas()`). Moreover, the geometry remeshing (refining and coarsening) functions as well as the data I/O are not parallelized, but are assumed to occur much less frequently than the actual fluid and reaction-advection-diffusion solvers. Therefore, since the fraction of sequential code is not negligible, the software should best be run on fast multi-core processors.

7.7 Tools, Dependencies and Documentation

A compiler with C++0x support (such as GCC 4.7 or higher) is required. The software depends on Boost (<http://www.boost.org>; 1.54.0 or higher), OpenMP, CMake (<http://www.cmake.org>) and vtk (<http://www.vtk.org/>; 5.8 or higher). The source code is extensively documented using Doxygen (<http://www.stack.nl/~dimitri/doxygen>). Git (<http://git-scm.com>) is used for version control. The software has only been tested on linux operating systems.

7.8 Availability

The documentation and source code are available on <http://tanakas.bitbucket.org/lbibcell/index.html>

8 Application Examples

8.1 Cell Division, Differentiation and Signalling

To demonstrate the capabilities of the software we first consider a tissue model with cell-type specific cell division and signalling-dependent differentiation (Figure 5). In the beginning, a circular cell with radius $R = 10$ is placed in the middle of a quadratic 400 by 400 domain (Figure 5A). Iso-pressure boundary conditions are set at the border of the domain. The initial cell is of red cell type, which is proliferating at a high rate. When considering a single layer epithelium, mass uptake, which is needed for modelling cell growth and finally proliferation, is assumed to occur from the apical cavity through the apical membrane. Additionally, the initial cell secretes a signalling factor $\mathcal{I}$ which inhibits differentiation of the red cell type into the green cell type. Once the cell area doubled, the cell is divided in a random direction (cf. Figure 5B). The daughter cells inherit the cell type, but only the mother cell continues to express the signalling molecule $\mathcal{I}$. All cells of red type integrate the concentration of $\mathcal{I}$ over their area. For low signalling levels, the red cell type differentiates into the green cell type. The green cell type does not grow and only divides if external forces stretch the cell. In Figure 5C, the daughter cell's signalling level dropped after cell division, and differentiation occurred. After several rounds of cell division, a tissue starts to form (cf. Figure 5D). The cells close to the secreting initial cell remain protected from differentiation, whereas more distant cells differentiate irreversibly. Due to the randomly chosen cell division axis, it might happen that the proliferating red cells get trapped (cf. Figure 5E). The expression of $\mathcal{I}$ is switched off at time $t=5000$, thus leading to complete differentiation shortly after (cf. Figure 5F). After proliferation stopped, the cells slowly rearrange because cell-cell junctions are bro-

ken if overly stretched, and new junctions are formed (according to Eq. (3)). At the boundary of the tissue, the cells try to reach a spherical shape, while in the middle mainly characteristic penta- and hexagonal shapes emerge (cf. Figure 5F and Supp. 10.4).

8.2 Turing Patterning on Growing Cellular Domains

To demonstrate the importance to investigate morphogenic signalling hypotheses on dynamically growing domains with cellular resolution, we solved a reaction-diffusion system, featuring the well-known diffusion-driven Turing instability ([41]), on a proliferating tissue. Figure 6A illustrates the interaction between a ligand L and its receptor R . Here, we assume that one ligand dimer molecule L binds to two receptors R , forming the complex R^2L which induces upregulation of the receptor on the membrane (e.g. [3]). Unbound receptor is turned over at a linear rate. The ligand can diffuse freely across the tissue and the entire domain, whereas the diffusion of the receptor is limited to a single cell's apical surface and is much slower. The dynamics can be formulated as a system of non-dimensional partial differential equations:

$$\partial_t R = \Delta R + \gamma (a - R + R^2 L) \quad (4)$$

$$\partial_t L = d \Delta L + \gamma (b - R^2 L) \quad (5)$$

where γ is a reactivity constant, a and b production constants, and d the relative diffusion coefficient of ligand and receptor. We note that the equations correspond to the classical Schnakenberg-type Turing mechanism ([11, 38]). It has previously been shown that such a receptor-ligand interaction can explain symmetry breaking in various morphogenetic systems ([21, 4, 1, 20, 40, 19]).

Depending on the type of domain we observe very different patterns. On a continuous domain we obtain the well known regular spot pattern (Figure 6B). On an idealized static cellular domain an overall regular pattern with irregular internal structure (Figure 6C) can be observed. Decreasing the simulation parameter γ , which inversely controls the distance between the spots, leads to even more unexpected patterns: for $\gamma = 100$, the local regularity is completely lost

(Figure 6D). Finally, on a dynamically growing cellular domain, where the local proliferation rate was set proportional to the R²L signal, we obtain irregular patterns (Figure 6E). For a lower value $\gamma = 100$, clusters of cells with high R²L signalling levels emerge (Figure 6F). In conclusion, even relatively simple signalling mechanisms can lead to significantly different results, depending on how the tissue is represented.

9 Discussion

We developed an extendible and open-source cell-based simulation environment, which is tailored to study morphogenetic problems. The novel framework permits the coupled simulation of a physically motivated visco-elastic cell model with regulatory signalling models. Processes such as viscous dissipation, elasticity, advection, diffusion, local reactions, local mass sources and sinks, cell division and cell differentiation are implemented. By applying our framework to Turing signalling systems we show that the signalling systems may behave very differently on dynamic tissues than on simple continuous tissue representations. We therefore advocate to test continuous morphogenetic signalling models on dynamically growing cellular domains.

The presented framework permits to study a variety of mechano-regulatory mechanisms. By making the cell division orientation dependent on signalling cues, the effect on the macroscopic tissue geometry may be studied. Cell migration can be modelled by introducing gradient-dependent forces on specific cell types. Cell sorting may be achieved by specifying multiple cell types with differential cell-cell junction strengths. The framework is specifically designed to study the mutual effects of signalling and biophysical cell properties.

The visco-elastic cell model represents cell shapes at very high resolution and is thus, unlike the vertex model, not restricted to densely packed tissues. Furthermore, hydrodynamic interaction, membrane tension and hydrostatic pressure are integral components of the model. The fact that a velocity field is available on the entire domain is a critical advantage to account for advection of the signalling com-

ponents, thus allowing for a spatial description of intracellular concentrations. The model is, however, not easily extendable to the third dimension. Since a meshing of the surface will be required, the algorithmic and computational complexity are expected to be significant and subject to future work. The presented framework is, however, ideal to study intrinsically two-dimensional morphogenetic problems, such as apical surface dynamics of epithelia as studied previously also by [8] and [17] in 2D.

Acknowledgement

Funding The authors acknowledge funding from the SNF Sinergia grant "Developmental engineering of endochondral ossification from mesenchymal stem cells" and the SNF SystemsX RTD NeurostemX.

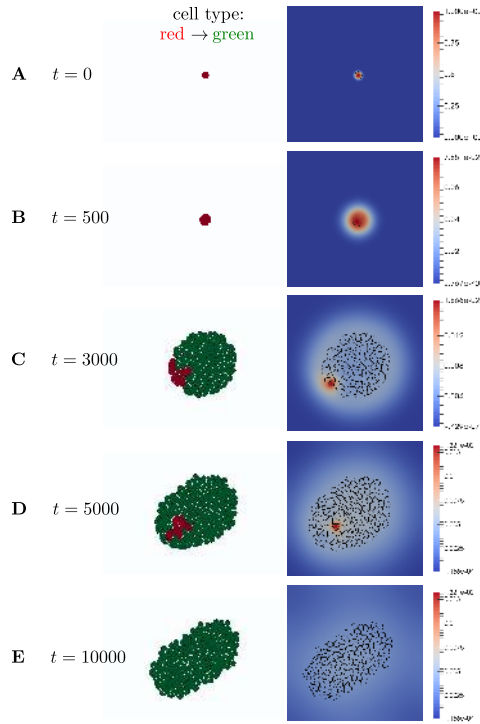


Figure 5: Cell Division, Differentiation and Signalling. **A** The initial configuration consists of a single, circular cell of type red. The red cell type proliferates at a high rate. The initial cell is tagged and expresses a signalling molecule $\mathcal{I}$ which inhibits differentiation. **B** The first cell division occurs. The division axis is chosen randomly. The daughter cell inherits the cell type from the mother cell, but only the mother cell keeps expressing the signalling molecule $\mathcal{I}$. **C** The signalling level (the spatially integrated concentration of the signalling molecule) drops in cells far away from the initial cell and differentiation into the green cell type occurs. The green cell type does not grow intrinsically, and only divides if overly stretched by external forces. **D** The highly proliferating red cells are trapped in the forming tissue due to the randomly chosen cell division axis. At $t = 5000$, the expression of the differentiation inhibiting molecule $\mathcal{I}$ is switched off, which leads to the differentiation of the remaining red cells. **E** In the absence of high proliferation, the cells rearrange to maximize the perimeter/area ratio. Characteristic penta- and hexagonal cell shapes emerge (cf. Supp. 10.4). Cells close to the boundary try to take a circular shape.

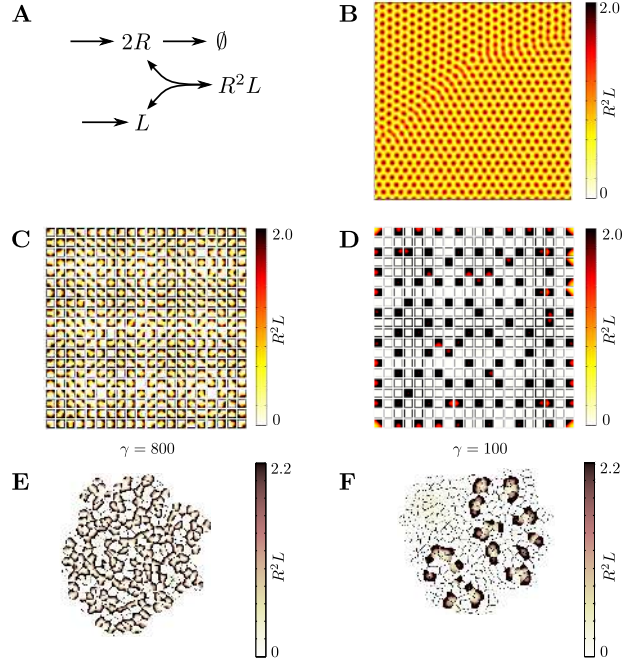


Figure 6: Turing Patterning on Growing Cellular Domains **A** Turing instability can be achieved by Schnakenberg-type reactions, involving a slowly diffusing compound R , here interpreted as a receptor, and a fast diffusing compound L , here interpreted as a freely diffusing ligand. One ligand molecule binds to two receptors, leading to the complex R^2L . The complex can be interpreted as a biological signal. **B** The model is solved on a continuous square lattice (using $d = 1$, $\gamma = 800$, $a = 0.1$, $b = 0.9$), resulting in the classical regular spot-pattern. The biological signal R^2L is shown. **C** The same system as in **B** is solved on an idealized static cellular domain, i.e. the diffusion of the receptor R is restricted to a cell. The emerging biological signal R^2L is now distributed irregularly. **D** The same system as in **C**, but with $\gamma = 100$, is solved on a idealized static cellular domain. Fewer cells show significant levels of signal R^2L and no regular pattern can be found (salt-and-pepper pattern). **E** The same system as in **C** is solved on a growing cellular domain. The proliferation rate of a cell is set proportional to its signal R^2L . The resulting pattern features regularity on a larger scale, but the local patterning significantly differs from the behaviour on continuous (**B**) and static cellular (**C**) domains. **F** The same system as in **D** is solved on a growing cellular domain. The proliferation rate of a cell is set proportional to the local intensity of the signal R^2L . Clusters of active cells with high levels of R^2L emerge.

References

- [1] Amarendra Badugu, Conradin Kraemer, Philipp Germann, Denis Menshykau, and Dagmar Iber. Digit patterning during limb development as a result of the BMP-receptor interaction. *Scientific reports*, 2:991, January 2012.
- [2] A. BARTOLONI, C. BATTISTA, S. CABASINO, P. S. PAOLUCCI, J. PECH, R. SARNO, G. M. TODESCO, M. TORELLI, W. TROSS, P. VICINI, R. BENZI, N. CABIBBO, F. MASSAIOLI, and R. TRIPICIONE. LBE SIMULATIONS OF RAYLEIGH-BÉNARD CONVECTION ON THE APE100 PARALLEL PROCESSOR. *International Journal of Modern Physics C*, 04(05):993–1006, October 1993.
- [3] Savério Bellusci, Yasuhide Furuta, Margaret G Rush, Randall Henderson, Glenn Winnier, and B L Hogan. Involvement of Sonic hedgehog (Shh) in mouse embryonic lung growth and morphogenesis. *Development (Cambridge, England)*, 124(1):53–63, January 1997.
- [4] Géraldine Cellière, Denis Menshykau, and Dagmar Iber. Simulations demonstrate a simple network to be sufficient to control branch point selection, smooth muscle and vasculature formation during lung branching morphogenesis. *Biology open*, 1(8):775–788, August 2012.
- [5] Shiyi Chen and Gary D Doolen. Lattice Boltzmann Methods for Fluid Flows. *Annual Review of Fluid Mechanics*, 30(1):329–364, January 1998.
- [6] Yeh-Shiu Chu, Sylvie Dufour, Jean Paul Thiery, Eric Perez, and Frederic Pincet. Johnson-Kendall-Roberts theory applied to living cells. *Physical review letters*, 94(2):028102, January 2005.
- [7] Dirk Drasdo, Stefan Hoehme, and Michael Block. On the Role of Physics in the Growth and Pattern Formation of Multi-Cellular Systems: What can we Learn from Individual-Cell Based Models? *Journal of Statistical Physics*, 128(1-2):287–345, April 2007.
- [8] Reza Farhadifar, Jens-Christian Röper, Benoit Aigouy, Suzanne Eaton, and Frank Jülicher. The influence of cell mechanics, cell-cell interactions, and proliferation on epithelial packing. *Current biology : CB*, 17(24):2095–104, December 2007.
- [9] Zhi-Gang Feng and Efstathios E Michaelides. The immersed boundary-lattice Boltzmann method for solving fluidparticles interaction problems. *Journal of Computational Physics*, 195(2):602–628, April 2004.
- [10] G Forgacs, R a Foty, Y Shafrir, and M S Steinberg. Viscoelastic properties of living embryonic tissues: a quantitative study. *Biophysical journal*, 74(5):2227–34, May 1998.
- [11] A Gierer and H Meinhardt. A theory of biological pattern formation. *Kybernetik*, 12(1):30–39, 1972.
- [12] François Graner and James Glazier. Simulation of biological cell sorting using a two-dimensional extended Potts model. *Physical Review Letters*, 69(13):2013–2016, September 1992.
- [13] Zhaoli Guo, Baochang Shi, and Chuguang Zheng. A coupled lattice BGK model for the Boussinesq equations. *International Journal for Numerical Methods in Fluids*, 39(4):325–342, June 2002.
- [14] Xiaoyi He, Qisu Zou, Li-shi Luo, and Micah Dembo. Analytic solutions of simple flows and analysis of nonslip boundary conditions for the lattice Boltzmann BGK model. *Journal of Statistical Physics*, 87(1-2):115–136, April 1997.
- [15] Stefan Hoehme and Dirk Drasdo. A cell-based simulation software for multi-cellular systems. *Bioinformatics (Oxford, England)*, 26(20):2641–2, October 2010.
- [16] Christian Huber, Andrea Parmigiani, Bastien Chopard, Michael Manga, and Olivier Bachmann. Lattice Boltzmann model for melting

- with natural convection. *International Journal of Heat and Fluid Flow*, 29(5):1469–1480, October 2008.
- [17] Shuji Ishihara and Kaoru Sugimura. Bayesian inference of force dynamics during morphogenesis. *Journal of theoretical biology*, 313:201–11, November 2012.
- [18] Qing Li, C.G. Zheng, N.C. Wang, and B.C. Shi. LBGK simulations of Turing patterns in CIMA model. *Journal of scientific computing*, 16(2):121–134, 2001.
- [19] Denis Menshykau, Pierre Blanc, Erkan Unal, Vincent Sapin, and Dagmar Iber. An interplay of geometry and signaling enables robust lung branching morphogenesis. *Development (Cambridge, England)*, October 2014.
- [20] Denis Menshykau and Dagmar Iber. Kidney branching morphogenesis under the control of a ligand-receptor-based Turing mechanism. *Physical biology*, 10:046003, 2013.
- [21] Denis Menshykau, Conrad Kraemer, and Dagmar Iber. Branch Mode Selection during Early Lung Development. *PLoS computational biology*, 8(2):e1002377, February 2012.
- [22] Roeland M H Merks, Michael Guravage, Dirk Inzé, and Gerrit T S Beemster. VirtualLeaf: an open-source framework for cell-based modeling of plant tissue growth and development. *Plant physiology*, 155(2):656–66, March 2011.
- [23] Nicolas Minc, David Burgess, and Fred Chang. Influence of cell geometry on division-plane positioning. *Cell*, 144(3):414–26, February 2011.
- [24] Rajat Mittal and Gianluca Iaccarino. Immersed Boundary Methods. *Annual Review of Fluid Mechanics*, 37(1):239–261, January 2005.
- [25] Timothy J Newman. Modeling Multicellular Systems Using Subcellular Elements. *Mathematical Biosciences and Engineering*, 2(3):613–624, August 2005.
- [26] X.D. Niu, C. Shu, Y.T. Chew, and Y. Peng. A momentum exchange-based immersed boundary-lattice Boltzmann method for simulating incompressible viscous flows. *Physics Letters A*, 354(3):173–182, May 2006.
- [27] a. Parmigiani, C. Huber, B. Chopard, J. Latt, and O. Bachmann. Application of the multi distribution function lattice Boltzmann approach to thermal flows. *The European Physical Journal Special Topics*, 171(1):37–43, May 2009.
- [28] Charles S Peskin. Numerical analysis of blood flow in the heart. *Journal of Computational Physics*, 25(3):220–252, November 1977.
- [29] Charles S. Peskin. The immersed boundary method. *Acta Numerica*, 11, July 2003.
- [30] Charles S. Peskin and Beth Feller Printz. Improved Volume Conservation in the Computation of Flows with Immersed Elastic Boundaries. *Journal of Computational Physics*, 105(1):33–46, March 1993.
- [31] Joe Pitt-Francis, Pras Pathmanathan, Miguel O Bernabeu, Rafel Bordas, Jonathan Cooper, Alexander G. Fletcher, Gary R. Mirams, Philip Murray, James M. Osborne, and Alex Walter. Chaste: A test-driven approach to software development for biological modelling. *Computer Physics Communications*, 180(12):2452–2471, December 2009.
- [32] S. Ponce Dawson, S Chen, and Gary D Doolen. Lattice Boltzmann computations for reaction-diffusion equations. *The Journal of Chemical Physics*, 98(2):1514, 1993.
- [33] Katarzyna a Rejniak. An immersed boundary framework for modelling the growth of individual cells: an application to the early tumour development. *Journal of theoretical biology*, 247(1):186–204, 2007.
- [34] Katarzyna A Rejniak, Harvey J Kliman, and Lisa J Fauci. A computational model of

- the mechanics of growth of the villous trophoblast bilayer. *Bulletin of mathematical biology*, 66(2):199–232, March 2004.
- [35] Simon Restrepo, Jeremiah J. Zartman, and Konrad Basler. Coordination of patterning and growth by the morphogen DPP. *Current Biology*, 24(6):R245–R255, 2014.
 - [36] S a Sandersius, M Chuai, C J Weijer, and Timothy J Newman. A ‘chemotactic dipole’ mechanism for large-scale vortex motion during primitive streak formation in the chick embryo. *Physical biology*, 8(4):045008, August 2011.
 - [37] S a Sandersius, Cornelis J Weijer, and Timothy J Newman. Emergent cell and tissue dynamics from subcellular modeling of active biomechanical processes. *Physical biology*, 8(4):045007, July 2011.
 - [38] J Schnakenberg. Simple chemical reaction systems with limit cycle behaviour. *Journal of theoretical biology*, 81:389–400, 1979.
 - [39] Maciej H Swat, Gilberto L Thomas, Julio M Belmonte, Abbas Shirinifard, Dimitrij Hmeljak, and James a Glazier. *Multi-scale modeling of tissues using CompuCell3D.*, volume 110. Elsevier Inc., January 2012.
 - [40] Simon Tanaka and Dagmar Iber. Interdependent tissue growth and Turing patterning in a model for long bone development. *Physical biology*, 10:56009, 2013.
 - [41] A M Turing. The Chemical Basis of Morphogenesis. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 237(641):37–72, August 1952.
 - [42] Junfeng Zhang, Paul C Johnson, and Alexander S Popel. An immersed boundary lattice Boltzmann approach to simulate deformable liquid capsules and its application to microscopic blood flows. *Physical biology*, 4(4):285–95, December 2007.

10 Supplementary Material

10.1 Lattice Boltzmann Method

For the fluid, the standard Lattice Boltzmann scheme (with the single-relaxation-time Bhatnagar-Gross-Krook collision operator) is used ([5]). It has been shown, that the incompressible Navier-Stokes equations can be recovered in the hydrodynamic limit ([5]). The Boltzmann equation is discretized on a D2Q9² lattice and reads:

$$f_i(\mathbf{x} + \mathbf{v}_i, t + 1) - f_i(\mathbf{x}, t) = -\frac{1}{\tau} (f_i(\mathbf{x}, t) - f_i^{eq}(\mathbf{x}, t)) + \Delta t \frac{w_i \rho}{c_s^2} (\mathbf{v}_i \cdot \mathbf{f}) \quad (6)$$

where f_i denotes the particle distribution function in the direction i , and can be interpreted as the probability density of finding particles with velocity $\mathbf{v}_i$ at time t at position $\mathbf{x}$. The relaxation time τ is related to the kinematic viscosity $\nu = c_s^2 (\tau - \frac{1}{2})$. For isothermal flows, the speed of sound c_s is defined as $c_s = c/\sqrt{3}$ using the lattice speed $c = \Delta x/\Delta t$. The lattice spacing and the time step are chosen as $\Delta x = \Delta t = 1$ to guarantee consistency with the lattice. The last term of Equation (6) represents the external body force ([14]).

Equation (6) implies a two step algorithm: firstly, the distribution functions f_i perform a free flight to the next lattice point (left hand side). Secondly, on each lattice point, the incoming distribution functions collide and relax towards a local equilibrium distribution f_i^{eq} , which is controlled by the relaxation time τ (right hand side). The equilibrium distribution is taken as:

$$f_i^{eq} = \omega_i \rho \left[1 + \frac{\mathbf{v}_i \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{v}_i \mathbf{v}_i - v^2) : \mathbf{u} \mathbf{u}}{2c_s^2} \right] \quad (7)$$

with the fluid velocity $\mathbf{u}$ and the fluid density ρ . The operator $:$ denotes the dyadic tensor scalar product.

²the nine-velocity lattice in two dimensions is defined as $\mathbf{v}_0 = [0, 0]$, $\mathbf{v}_i = [\cos(\pi(i-1)/2), \sin(\pi(i-1)/2)]$ for $i = \{1, 2, 3, 4\}$, and $\mathbf{v}_i = \sqrt{2} [\cos(\pi(i-1)/2) + \pi/4, \sin(\pi(i-1)/2) + \pi/4]$ for $i = \{5, 6, 7, 8\}$

For the D2Q9 lattice, the population weights ω_i can be found as $\omega_0 = 4/9$, $\omega_{1-4} = 1/9$, and $\omega_{5-8} = 1/36$.

Finally, the macroscopic quantities (density ρ and momentum density $\rho \mathbf{u}$) can be computed using the zeroth and first order moments:

$$\rho = \sum_{i=0}^8 f_i \quad \rho \mathbf{u} = \sum_{i=0}^8 f_i \mathbf{v}_i \quad (8)$$

The fluid pressure p is related to the mass density ρ as $p = \rho c_s^2$.

For solving the reaction-advection-diffusion equation of a compound ϕ , a multi-distribution function approach is chosen ([2, 13]), i.e. an additional Lattice Boltzmann solver is coupled to the fluid solver. We implemented different schemes (D2Q4, D2Q5), which exhibit slightly different stability and accuracy ([18]). For the D2Q5³ scheme, we follow [16] and [27] and write for the Lattice Boltzmann equation:

$$g_i(\mathbf{x} + \mathbf{v}_i, t + 1) - g_i(\mathbf{x}, t) = -\frac{1}{\tau_D} (g_i(\mathbf{x}, t) - g_i^{eq}(\mathbf{x}, t)) \quad (9)$$

The equilibrium distribution functions is taken as:

$$g_i^{eq} = \phi w_i \left[1 + \frac{1}{c_s^2} (\mathbf{v}_i \cdot \mathbf{u}) \right] \quad (10)$$

Instead of the local first moment, the velocity field $\mathbf{u}$ from the fluid is transferred. The weights w_i are chosen as $w_0 = 1/3$ and $w_{1-4} = 1/6$. The relaxation parameter τ_D is related to the diffusion coefficient as $D = c_s^2 (\tau_D - \frac{1}{2})$, and the local compound density ϕ reads:

$$\phi = \sum_{i=0}^4 g_i \quad (11)$$

All variables are expressed in Lattice Boltzmann units $\delta x, \delta t$ and have to be converted to physical units.

³ the five-velocity lattice in two dimensions is defined as $\mathbf{v}_0 = [0, 0]$ and $\mathbf{v}_i = [\cos(\pi(i-1)/2), \sin(\pi(i-1)/2)]$ for $i = \{1, 2, 3, 4\}$

10.1.1 No-flux Boundary Condition for Reaction-Advection-Diffusion Equations

The missing incoming distribution functions are approximated by equilibrium distributions. The momentum is spatially first order interpolated between the fluid in direction of the missing population, and the known zero momentum at the wall. The density is spatially first order extrapolated from the fluid.

10.1.2 Pressure Boundary Condition for the Fluid Equations

Within the computational domain, lattice points can act as internal pressure boundaries, i.e. points with prescribed fluid pressure, which may be needed to implement mass sinks in case of growing cells/tissues. All distribution functions are rescaled such that the prescribed pressure is obtained. The velocity field is not affected, since the rescaling factor γ cancels:

$$\mathbf{u}(\mathbf{x}, t) = \frac{\sum_i \gamma \mathbf{v}_i f_i(\mathbf{x}, t)}{\sum_i \gamma f_i(\mathbf{x}, t)} \quad (12)$$

10.2 Immersed Boundary Method

The PDE's are preferentially solved on a Eulerian grid, whereas cells are naturally represented in a Lagrangian manner. The two frames can be bridged using the immersed boundary method, which was introduced in [28]. This technique is appropriate for complex and moving boundaries and fluid-structure interactions. The material equations are solved on a Eulerian grid, whereas the boundary equations are expressed in a Lagrangian way. An introduction can be found in [24], and detailed discussions in [29].

The curvilinear (boundary) coordinates (q, r, s) are attached to a material point. At time t , the coordinates in the Eulerian framework read $\mathbf{X}(q, r, s, t)$. The material derivative $\frac{D\mathbf{u}}{Dt}(\mathbf{x}, t) = \frac{\partial^2 \mathbf{X}}{\partial t^2}(q, r, s, t)$ is the acceleration of whatever material point is at position $\mathbf{x}$ at time t . The conversion from Lagrangian variables (e.g. mass density $M(q, r, s, t)$ or force density $\mathbf{F}(q, r, s, t)$) to their Eulerian counterparts is ex-

ecuted by integrating over the material coordinates:

$$\rho(\mathbf{x}, t) = \int M(q, r, s, t) \delta(\mathbf{x} - \mathbf{X}(q, r, s, t)) dq dr ds \quad (13)$$

$$\mathbf{f}(\mathbf{x}, t) = \int \mathbf{F}(q, r, s, t) \delta(\mathbf{x} - \mathbf{X}(q, r, s, t)) dq dr ds \quad (14)$$

where $\delta(\cdot)$ denotes the delta Dirac function. For the numerical implementation, the Delta Dirac function is approximated in such a way that it covers multiple grid points, on which the conversions in Equation (13) and (15) are evaluated. The equation of motion of the Lagrangian particles reads:

$$\begin{aligned} \frac{\partial \mathbf{X}}{\partial t}(q, r, s, t) &= \\ \mathbf{u}(\mathbf{X}(q, r, s, t), t) &= \int \mathbf{u}(\mathbf{x}, t) \delta(\mathbf{x} - \mathbf{X}(q, r, s, t)) d\mathbf{x} \end{aligned} \quad (15)$$

Using the Hodge decomposition⁴, the material is described by

$$\rho \left[\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right] = -\nabla p + \mu \Delta \mathbf{u} + \mathbf{f} \quad (16)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (17)$$

To couple the Immersed Boundary method to the Lattice Boltzmann method, the LB forcing term $\mathbf{f}$ has to be obtained from the Lagrangian force $\mathbf{F}$ following a similar way as [9]. The Lagrangian force $\mathbf{F}_i$ of the geometry point i is composed of membrane forces (cf. Equation (2)) and cell junction forces (cf. Equation (2)):

$$\mathbf{F}_i = \mathbf{F}_i^m + \mathbf{F}_i^{cj} \quad (18)$$

$\mathbf{F}_i$ is distributed to the local fluid neighborhood according to Equation (14), and the Eulerian force $\mathbf{f}$ on the lattice point $\mathbf{x}$ is then used in the LB collision term in Equation (6).

The discretized delta Dirac function for one spatial dimension is defined as:

$$\delta^{1D}(\mathbf{r}) = \begin{cases} \frac{1}{4} \left(1 + \cos \left(\frac{\pi \|\mathbf{r}\|}{2} \right) \right) & \text{if } \|\mathbf{r}\| \leq 2 \\ 0 & \text{if } \|\mathbf{r}\| > 2 \end{cases} \quad (19)$$

⁴any vector field $\mathbf{u}$ can be written as $\rho \frac{D\mathbf{u}}{Dt} - \mathbf{f} = -\nabla p + \mathbf{w}$, where $\nabla \cdot \mathbf{w} = 0$

and the two-dimensional discretized delta Dirac function is a multiplication of Equation (19):

$$\delta(\mathbf{r}) = \delta^{1D}(\mathbf{r}) \delta^{1D}(\mathbf{r}) \quad (20)$$

This implies that the Lagrangian force $\mathbf{F}_i$ of a membrane point i is distributed to the $4 \times 4 = 16$ nearest fluid lattice points. By discretizing the membrane into membrane points i , the discretized delta Dirac functions overlap.

10.3 Validations

To validate the implementation of the fluid solver, the reaction-advection-diffusion solver and the immersed boundary method, validation tests using problems with known analytical solutions are executed.

The dimensionless problem is as follows: a circular cell (diameter 1) is placed in the middle of a square domain of size 10 by 10. The end simulation time is $T = 1$. Inside the cell, a diffusing species is defined, where the cell membrane acts as a no-flux boundary condition. No production and degradation of the species occurs. The initial condition of the species is set to be uniformly 1. The diffusion coefficient as well as the kinematic viscosity of the fluid are set to 1. No membrane forces are applied; iso-pressure out-flow boundary conditions are applied to the domain boundaries. Please see *movie_S3.avi* for an impression of how the concentration decreases as the area increases.

Three different scenarios are considered:

- Case I: a constant point mass source is placed in the middle of the cell (dimensionless strength is 1); see Fig. 7A
- Case II: a constant uniform mass source inside the cell (dimensionless strength is $\log(2)/A_0$, where $A_0 = 0.5^2\pi$ is the initial cell area; this leads to a doubling of the cell area in time T), see Fig. 7B
- Case III: a uniform mass source inside the cell, where the strength is scaled linear proportional to the species concentration ($= 1 \cdot C(t)$, where $C(t)$ is the concentration of the species; this

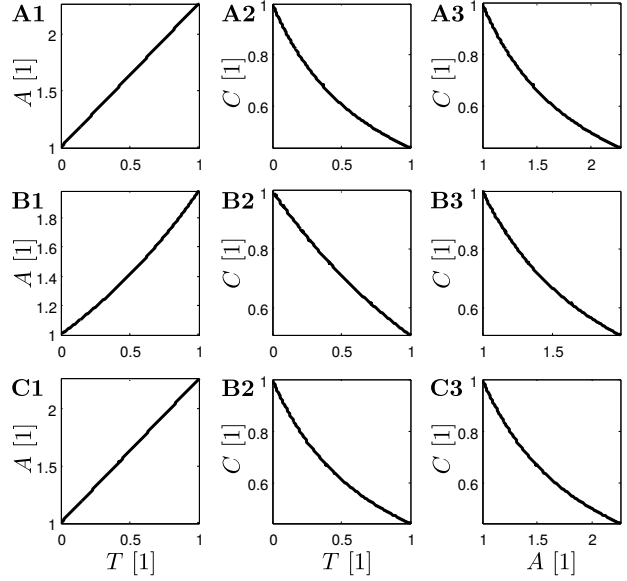


Figure 7: **Validation Cases: Evolution of Area and Concentration** T is the non-dimensional normalized time, A the non-dimensional normalized area, C the non-dimensionalized normalized concentration. The column **X1** shows area A vs. time T ; column **X2** shows concentration C vs. time T ; and column **X3** shows concentration C vs. area A . **A** Case I: constant point-like mass source; **B** Case II: constant uniform mass source; **C** Case III: uniform mass source proportional to the concentration C .

leads to a linear mass/area increase in time), see Fig. 7C

For all three cases, the circular cell starts to grow, and the species is diluted, i.e. the species concentration drops as the area increased. Ideally, the integrated species concentration is conserved. The relative errors of the simulations w.r.t. the analytical solutions are computed for both the area and concentration evolution.

The simulations are executed at different levels of space and time discretization. As a measure, we take the diameter resolution of the circular cell, which is set to $\{10, 20, 40, 80\}$. As a result, the discretized values (LB denotes Lattice Boltzmann units, and ND

denotes non-dimensional) for all cases can be found in Table 1.

circle diameter	10	20	40	80
domain size	100x100	200x200	400x400	800x800
τ fluid	1	1	1	1
ν^{LB} fluid	1/6	1/6	1/6	1/6
τ diff	1	1	1	1
ν^{LB} diff	1/6	1/6	1/6	1/6
δ_x	0.1	0.05	0.025	0.0125
T^{LB}	1e4	4e4	1.6e5	6.4e5
δ_t	1e-4	25e-6	625e-8	15625e-10
$S^{LB} = S^{ND} \cdot \frac{\delta_t}{\delta_x^2}$	1e-2	1e-2	1e-2	1e-2

Table 1: Lattice-Boltzmann discretization of Cases I-III. ND and LB denote non-dimensional and Lattice Boltzmann units, respectively

The validation results for Case I are given in Figure 8. Figure 8A shows the relative error temporal evolution of the cell area for the different resolutions, and 8B the relative error temporal evolution of the integrated concentration. The concentration error is much more noisy because it is only evaluated on lattice points. When the cell grows, the membrane sweeps across the lattice, and whenever a new lattice site enters the intracellular domain, the integrated concentration increases abruptly. To compute the convergence w.r.t. to spatial and temporal resolution, the relative errors at $T^{ND} = 1$ are evaluated for the area, but the temporal mean is taken for the concentration convergence analysis. Figure 8C shows the convergence analysis for the area. For the finer grids, the convergence order is approximately 1. Figure 8D depicts the convergence analysis for the integrated concentration, where the fitted order of convergence is ≈ 1.2 .

The validation results for Case II are given in Figure 9. Figure 9A shows the relative error temporal evolution of the cell area for the different resolutions, and 9B the relative error temporal evolution of the integrated concentration. The concentration error is much more noisy because the it is only evaluated on lattice points. When the cell grows, the membrane sweeps across the lattice, and whenever a new lattice site enters the intracellular domain, the integrated concentration increases abruptly. To compute the convergence w.r.t. to spatial and temporal resolution, the relative errors at $T^{ND} = 1$ are evaluated for

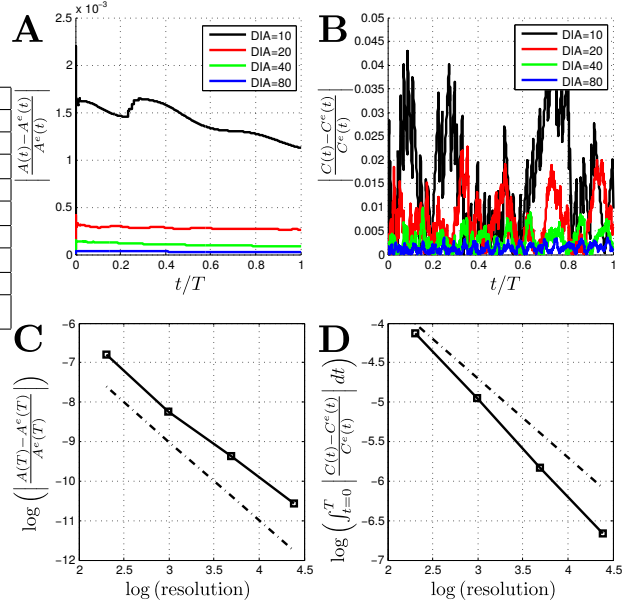


Figure 8: **Case I Validation: Point Source.** An initially circular cell is growing through addition of mass. The mass is added by using a point source in the middle of the domain. Four different spatial resolutions have been computed, corresponding to a resolution of $DIA = \{10, 20, 40, 80\}$ lattice points for the diameter of the initial circular cell. **A** The time evolution of the relative error of the cell area for 4 different lattice resolutions. **B** The time evolution of the relative error of the spatially integrated concentration (mass conservation) for 4 different lattice resolutions. **C** Convergence plot of the relative error of the cell area as a function of the lattice resolution. The fitted slope is -1.80. The dashed line denotes a slope of -2. **D** Convergence plot of the relative error of the integrated concentration (mass conservation) as a function of the lattice resolution. The fitted slope is -1.22. The dashed line denotes a slope of -1.

the area, but the temporal mean is taken for the concentration convergence analysis. Figure 9C shows the convergence analysis for the area. For the finer grids, the convergence order is approximately 1. Figure 9D depicts the convergence analysis for the integrated concentration, where the fitted order of convergence

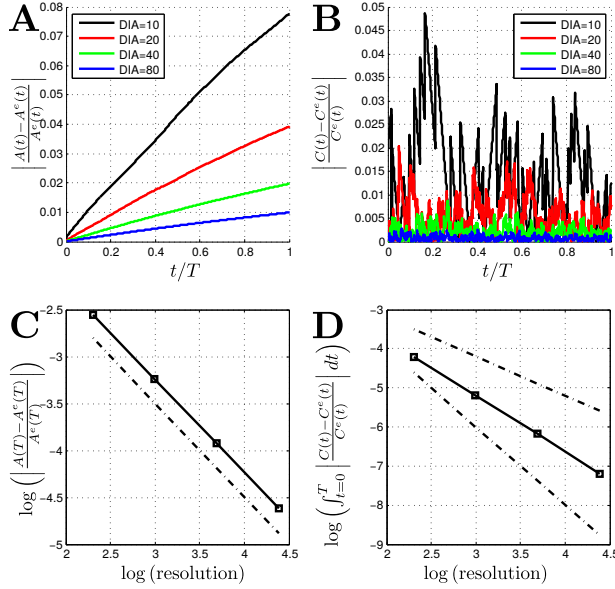


Figure 9: Case II Validation: Uniformly Distributed Source. An initially circular cell is growing through addition of mass. The mass is added by applying a constant uniform mass source on the entire cellular domain. Four different spatial resolutions have been computed, corresponding to a resolution of $\text{DIA} = \{10, 20, 40, 80\}$ lattice points for the diameter of the initial circular cell. **A** The time evolution of the relative error of the cell area for 4 different lattice resolutions. **B** The time evolution of the relative error of the spatially integrated concentration (mass conservation) for 4 different lattice resolutions. **C** Convergence plot of the relative error of the cell area as a function of the lattice resolution. The fitted slope is -0.99. The dashed line denotes a slope of -1. **D** Convergence plot of the relative error of the integrated concentration (mass conservation) as a function of the lattice resolution. The fitted slope is -1.43. The dashed lines denote slopes of -1 and -2, respectively.

is ≈ 1.4 .

The validation results for Case III are given in Figure 10. Figure 10A shows the relative error temporal evolution of the cell area for the different resolutions, and 10B the relative error temporal evolution

of the integrated concentration. The concentration error is much more noisy because it is only evaluated on lattice points. When the cell grows, the membrane sweeps across the lattice, and whenever a new lattice site enters the intracellular domain, the integrated concentration increases abruptly. To compute the convergence w.r.t. to spatial and temporal resolution, the relative errors at $T^{ND} = 1$ are evaluated for the area, but the temporal mean is taken for the concentration convergence analysis. Figure 10C shows the convergence analysis for the area. For the finer grids, the convergence order is approximately 1. Figure 10D depicts the convergence analysis for the integrated concentration, where the fitted order of convergence is ≈ 1.5 .

Finally, the immersed boundary force distribution is validated (Case IV). The initial setup is similar to the Cases I-III, but a constant membrane force is applied, and there is no mass source. Due to numerical leakage, the cell starts to shrink and loses mass/area. The dimensionless force is chosen to be $F^{ND} = 1e3$, and the dimension conversion is described in Table 2. The discretization of the validation setup can be found in Table 3.

	ND	LB
mass density	ρ^{ND}	ρ^{LB}
mass units $[M]$	[1]	$[\delta_m^{LB \rightarrow ND}]$
mass density units 2D $[M/L^2]$	[1]	$\left[\frac{\delta_m^{LB \rightarrow ND}}{(\delta_x^{LB \rightarrow ND})^2} \right]$
force	F^{ND}	F^{LB}
force units $[ML/T^2]$	[1]	$\left[\frac{\delta_m^{LB \rightarrow ND} \delta_x^{LB \rightarrow ND}}{(\delta_t^{LB \rightarrow ND})^2} \right]$

Table 2: Lattice-Boltzmann discretization of mass and force. ND and LB denote non-dimensional and Lattice Boltzmann units, respectively.

Besides the spatial and temporal discretization of the lattice, also the discretization of the immersed boundary is studied in Case IV. The distance between any two neighboring boundary points is shorter than the parameter **MAXLENGTH**, which is always a factor of the lattice discretization δ_x . If **MAXLENGTH** is set to 0.5, then the distance between two boundary points does not exceed half of the lattice spacing. **MAXLENGTH=0.5** is a frequently used constant ([30]); however, here, we also tested smaller (0.1) and larger

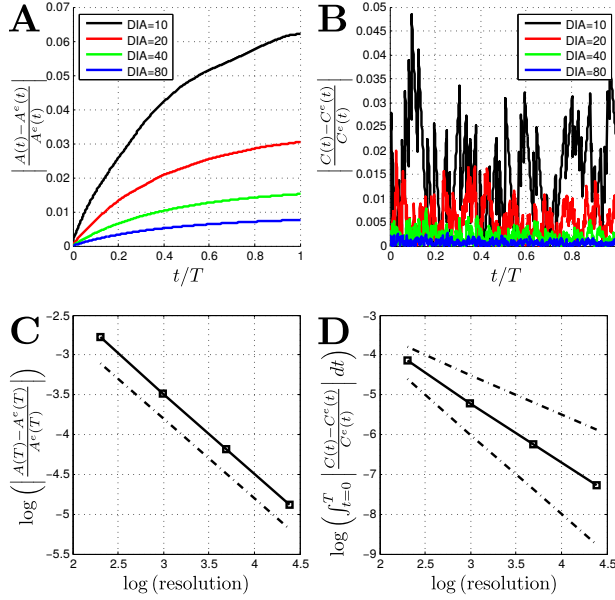


Figure 10: **Case III Validation: Fully Coupled System.** An initially circular cell is growing through addition of mass. The mass is added by applying a mass source on the entire cellular domain, but the mass source strength is proportional to the concentration of the intracellular concentration. Therefore, the cellular area should increase linearly in time. Four different spatial resolutions have been computed, corresponding to a resolution of $\text{DIA} = \{10, 20, 40, 80\}$ lattice points for the diameter of the initial circular cell. **A** The time evolution of the relative error of the cell area for 4 different lattice resolutions. **B** The time evolution of the relative error of the spatially integrated concentrations (mass conservation) for 4 different lattice resolutions. **C** Convergence plot of the relative error of the cell area as a function of the lattice resolution. The fitted lattices is -1.01. The dashed line denotes a slope of -1. **D** Convergence plot of the relative error of the integrated concentration (mass conservation) as a function of the lattice resolution. The fitted slope is 1.50. The dashed lines denote slopes of -1 and -2, respectively.

(2.5) values.

The validation results of Case IV are showed in

circle diameter	80	160	320	640
domain size	100x100	200x200	400x400	800x800
τ fluid	1	1	1	1
ν^{LB} fluid	1/6	1/6	1/6	1/6
τ diff	1	1	1	1
ν^{LB} diff	1/6	1/6	1/6	1/6
δ_x	0.1	0.05	0.025	0.0125
T^{LB}	1e4	4e4	1.6e5	6.4e5
δ_t	1e-4	2.5e-5	6.25e-6	1.5625e-06
mass density $\rho^{ND} = 1$	1	1	1	1
$\delta_m = \frac{\rho^{ND}}{\rho^{LB}} (\delta_x)^2$	1e-2	2.5e-3	6.25e-4	1.5625e-4
$F^{LB} = F^{ND} \frac{\delta_t^2}{\delta_m \delta_x}$	1e-3	5e-4	2.5e-4	1.25e-4

Table 3: Lattice-Boltzmann discretization of the convergence analysis of Case IV. ND and LB denote non-dimensional and Lattice Boltzmann units, respectively.

Figure 11. In Figure 11A, the evolution of the relative errors of the area are shown for different lattice resolutions. Figure 11B shows the convergence as a function of MAXLENGTH. The $\text{MAXLENGTH} = \{0.1, 0.5\}$ values lead to almost similar results, thus confirming that $\text{MAXLENGTH} = 0.5$ is indeed sufficient. The convergence as a function of the lattice resolution is shown in Figure 11C. The fitted order of convergence is approximately 1. $\text{MAXLENGTH} = \{0.1, 0.5\}$ lead to indistinguishable results; however, even $\text{MAXLENGTH} = 2.5$ still leads to converging results.

Here, for the convergence analyses, the fluid, mass sources, fluid outlet boundary conditions, the advection and diffusion of a species, its no-flux boundary condition, the immersed boundary and membrane forces are considered. We showed that all aspects are converging and conclude that the net order of convergence of the fully coupled system is 1. Depending on the aspect under consideration, it can be higher. It is well known that the standard Immersed Boundary method suffers from fluid leakage, because the discretized kernel function is not divergence-free. Several improvements have been proposed (e.g. [30, 26]).

10.4 Comparison to Farhadifar et al., 2007

The cell topologies are compared to results of the cell vertex model as presented in [8]. Starting from a single cell, a tissue with more than 1000 cells is grown

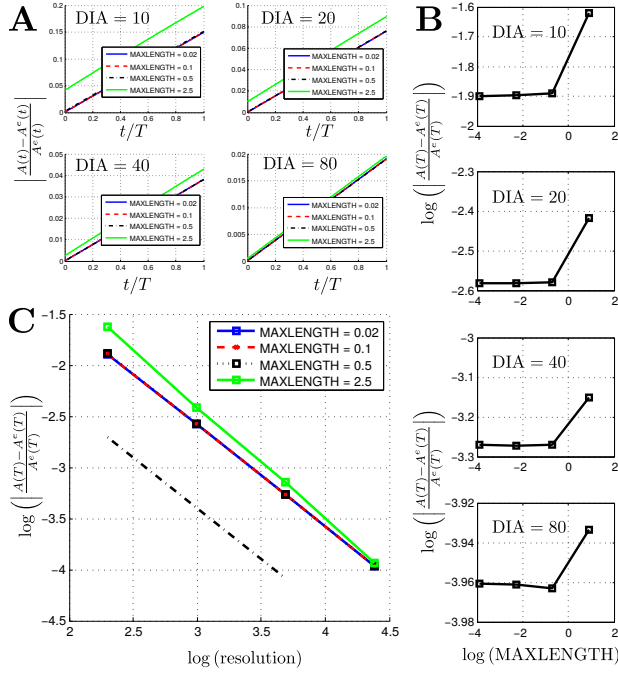


Figure 11: Case IV Validation: Mass Conservation A constant membrane tension is applied to an initially circular cell. The contractile force will lead to a shrinking cell due to mass leaking of the immersed boundary. Here, the influence of the immersed boundary discretization is investigated. Four different spatial resolutions have been computed, corresponding to a resolution of $\text{DIA} = \{10, 20, 40, 80\}$ lattice points for the diameter of the initial circular cell. **MAXLENGTH** denotes the maximally tolerated distance between each two immersed boundary points (measured in space discretization units). **A** The time evolution of the relative error of the cell area for 4 different lattice resolutions, and for 3 values of **MAXLENGTH**. **B** The relative error of the cell area as a function of **MAXLENGTH** for different lattice resolutions. **C** Convergence plot of the relative error of the cell area as a function of the lattice resolution, plotted for 3 values of **MAXLENGTH**. The fitted slope of the two finest lattices is 0.9891.

with a constant mass source of 0.004 (in LB units). A cell is divided in a random direction if it exceeds

an area of 380 (in LB units). The membrane tension is varied ($F^{LB} = \{0.001, 0.01, 0.02\}$ in LB units).

The snapshots in Figure 12 show the topology for different values of membrane tension (**A** very low membrane tension; **B** medium membrane tension; **C** high membrane tension). The higher the membrane tension, the higher the rearrangement activity because cell-cell junctions are broken more easily, and the cells can more easily assume a preferential shape (hexagonal in the domain, circular at the boundary). In Figure 12D, the tissue shown in figure 12B is relaxed, i.e. cell division is stopped, and only rearrangement is occurring towards an equilibrium configuration.

For these cases, the number of vertices are counted. The result is shown in Figure 13, together with the experimental and model results of [8]. Although the topologies for different membrane tensions differ significantly, the distribution of the number of vertices (equivalent to the number of neighbors) is not significantly affected. The relaxed topology, however, is shifted towards higher occurrence of hexagons. Overall, our results are in agreement with the experimental and simulation results obtained by [8].

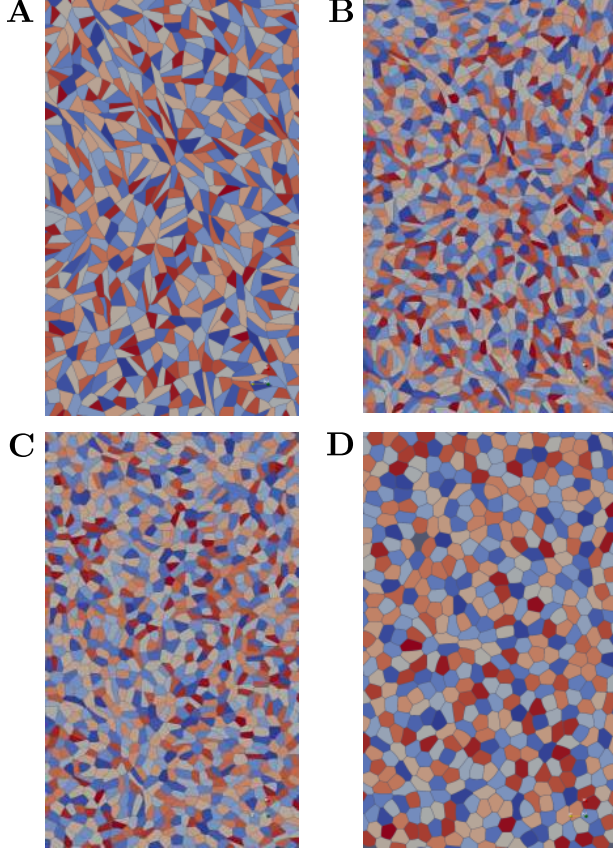


Figure 12: **Cell Topologies for Different Membrane Forces.** **A** The membrane force constant is small ($F^{LB} = 0.001$), such that the cells cannot rearrange adequately. **B** The membrane force constant is relatively high ($F^{LB} = 0.01$), such that the rearrangement activity alters the topology significantly. **C** The membrane force constant is even higher ($F^{LB} = 0.02$). **D** The tissue shown in **B** is relaxed towards equilibrium. The color code denotes the domain identifier value (red = young cells, blue = old cells)

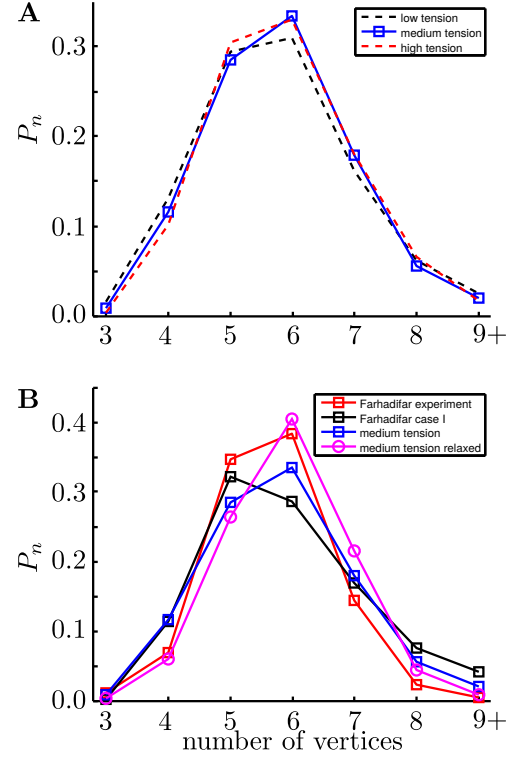


Figure 13: **Comparison of the Cell Topologies to Literature Results.** P_n denotes the occurrence fraction of an n -sided polygon. Polygons with $n > 9$ are lumped into $n = 9$. **A** 'Low/medium/high tension' correspond to different membrane tensions ($F^{LB} = \{0.001, 0.01, 0.02\}$ in LB units). 'high tension' has a slightly higher occurrence of penta- and hexagons. **B** 'Farhadifar experiment' is the polygon distribution in a third instar *Drosophila* wing disc. 'Farhadifar case I' and 'Farhadifar case II' denote simulations with low and high relative contractility, respectively. 'Medium tension relaxed' corresponds to an equilibrated simulation with $F^{LB} = 0.01$ after proliferation has stopped.